

Computer Graphics Project 3: Designing a BVH Viewer for Motion-Related Applications

Handed out: May 19, 2026

Due: 23:59, June 7, 2026 (NO SCORE for late submissions!)

- LMS course home > Weekly Learning (주차학습) > Week 11> Project 2
- Compress your files into a zip file as in the following example. The zip file can be named whatever you want.

```
+ submission.zip
- main.py
- ...
- report.pdf
```

- Your program may consist of several python source files. But the main module should be in **main.py**. That is, your program should be executed with the following command:

```
python main.py
```

1. In this project, you will build a **BVH viewer** that visualizes and animates motion capture data. You will then **extend your viewer with one of your own motion-related ideas or applications**.

A. This project is not only about implementation, but about:

- i. **designing systems for motion visualization and interaction**
- ii. **exploring your own ideas and applications related to motion**
- iii. **explaining and justifying your design choices**

B. Your implementation will extend your viewers from Project 1 and Project 2.

2. Program Requirements

A. Core Motion Viewer

- i. Your viewer should:

1. Parse BVH files of your choice

A. Since BVH files may use different coordinate scales (length 1 might correspond to 1 meter, 1 centimeter, 1 inch, etc.), you may apply appropriate

scaling to the loaded skeleton for better motion visualization.

- B. You may download publicly available BVH files from online motion capture databases and visualize and use them in your viewer and motion-related applications.

Examples:

- i. <http://motion.hahasoha.net/>
- ii. <http://mocap.cs.sfu.ca/>
- iii. <http://dancedb.eu/main/performances>
- iv. You are encouraged to try different types of motion data from various online motion capture datasets beyond the examples above.

2. Animate skeletal motion over time

3. Visualize animated motion clearly

- A. You may choose your own representation and rendering styles for joints and links.

- i. Examples: lines, boxes, cylinders, spheres, custom geometry

4. Provide basic camera/viewer interaction

ii. In your **report**:

1. explain how your viewer visualizes skeletal motion and why you chose that representation
2. explain your interaction design (keyboard, mouse, etc.) and why you designed it that way
3. specify the BVH files used in your project, including the source dataset name and a brief description of the motion (e.g., walking, dancing, martial arts, etc.)

B. Motion Application / Exploration

- i. After building the viewer, **extend your viewer** with **one motion-related idea or application of your choice**.

1. The motion application / exploration should be integrated into your viewer system.

ii. Examples may include, but are not limited to:

1. motion trajectory visualization
 2. multiple synchronized characters
 3. motion-aware camera systems
 4. procedural motion modification
 5. stylized rendering
 6. motion editing
 7. interaction between multiple animated characters
 8. game-like character control
 9. combinations of the above ideas
 10. any other motion-related idea you want to explore
- iii. In your **report**:
1. explain what motion-related idea or application you explored
 2. explain how your exploration/application works
 3. explain why you chose that idea or what you found interesting about it

3. Video Requirements

- A. A **hyperlink to a video** demonstrating your program, uploaded to an online video streaming service (e.g., YouTube, Vimeo).
- i. The video is intended to demonstrate your system and motion application, not to provide a detailed explanation.
 - ii. The video must clearly demonstrate:
 - skeletal motion playback
 - camera/viewer interaction
 - your motion application or exploration
 - iii. Explanations or annotations are optional.
- B. **The uploaded video MUST be publicly accessible.**
- C. The video should be **no longer than 20 seconds**.

4. Report Requirements

- A. The report should include:
 - i. Core Motion Viewer report requirement (2-A-ii)
 - ii. Motion Application / Exploration report requirement (2-B-iii)
 - iii. Video link (3-A)
- B. The report should be concise (approximately 2-5 pages). Clarity is more important than length.
- C. You may write the report in either English or Korean. There is no advantage or disadvantage based on the language you choose.

5. What to submit: A zip file including

A. Python source files (.py)

- i. Your program may consist of several python source files. But the main module should be in **main.py**.

B. A report (.pdf)

- i. Use provided "Project3-report-template(en).docx" or "Project3-report-template(kr).docx"

C. BVH files and additional assets used in your project

- i. **You must include the BVH files (and OBJ files if custom geometry is used) required to run your project.**
- ii. Use **relative paths in your source code** so that the program runs correctly on the TA's computer.
- iii. The "relative paths" in the source code should not be platform-dependent. **Use `os.path.join()` for platform-independent path joining.**

D. Note:

- The video must be accessible via a hyperlink included in the report.
- If the video link is missing or not accessible, it may be treated as missing and result in a penalty.

6. **Runtime** Requirements

- A. You must implement all requirements in a single program. This project DOES NOT require each requirement to be a separate program.
- B. The window size doesn't need to be (800, 800). Use a sufficiently large window to clearly visualize motion and viewer details.
- C. **Your program must use OpenGL 3.3 Core Profile**, meaning that ...

- i. Your python code must include:

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3)
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3)
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE)
```

- ii. Your shader code must start with:

```
#version 330 core
```

- D. **Your program should be able to run on Python 3.10 with only NumPy, PyOpenGL, glfw, PyGLM, Pillow installed. Do not use any other additional python modules.**
- E. Only **glfw** is allowed for event processing and window & OpenGL context management. **Do not use glut functions for this purpose.**
- F. **If your program does not meet these requirements, it may not run on the TA's computer, which may result in a significant penalty.**

7. **Evaluation** will be primarily based on:

- A. your motion viewer design and implementation (as required in 2-A-ii)
- B. your motion application / exploration (as required in 2-B-iii)
- C. how clearly your video demonstrates your program
- D. Note:
 - The video and report are essential components of this project.
 - Insufficient or missing video/report may result in significant penalties.
 - Strong emphasis is placed on how clearly you explain your design choices.

8. Additional Bonus Points

- A. Additional bonus points (up to **+20 points**) may be awarded for particularly creative, technically impressive, or highly polished motion applications / explorations.

9. Additional information

- A. Python provides powerful string methods helpful for parsing a bvh file. Among them, `split()` will be most useful.